BUIDCO Project Monitoring Dashboard

Presentation notes — project overview, architecture, code flow and likely questions

1. What the project is

A web dashboard that lets **BUIDCO** (Bihar Urban Infrastructure Development Corporation) track every urban infrastructure project in one place — from sanction, through tendering and construction, to O&M handover.

It replaces scattered Excel sheets with a single live system that senior management, project directors and office staff all read from.

The problem it solves

- Project data lived in many Excel files and emails — no single source of truth.
- Management had no live view of progress, delays or money spent.
- No record of who changed what, so data could not be trusted or audited.
- Expiring bank guarantees and O&M contracts were easy to miss.

Project at a glance

23	11	14	21	4
DB TABLES	API GROUPS	SERVICES	UI PAGES	USER ROLES

2. Key features

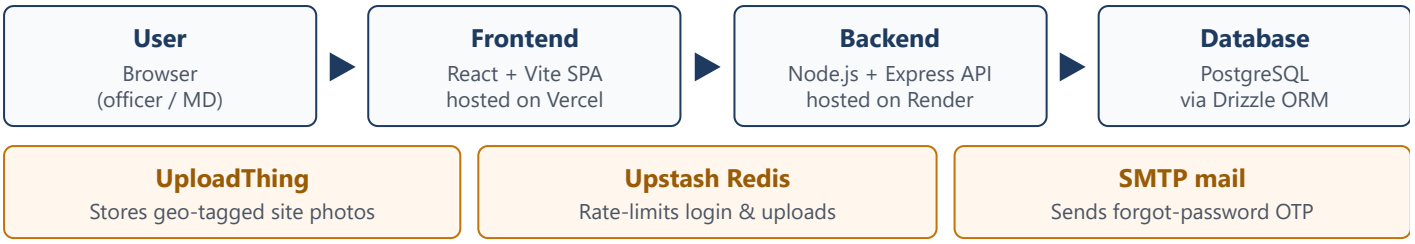
Overview dashboard	Live KPIs — total, completed, in progress, delayed, on hold — plus charts and expiry alerts.
Projects register	One searchable, filterable, sortable table of every project. Columns can be shown/hidden and exported to Excel, PDF or PowerPoint.
Input Sheet	One large form to create or edit a project, split into sections (Basic Info, Phase & Dates, Progress, Contract, O&M, Funding…).
Excel import	Bulk-create projects from a downloadable template. Every row is validated before anything is saved.
CoS / EoT	Tracks Change of Scope and Extension of Time requests against each project.
Management Actions	Action items raised against projects, with owners and status.
Funds & UC	Fund releases and Utilisation Certificates, split by central and state share.
Pre-Monsoon Prep	Seasonal readiness checklist for drainage and flood-prone work.
Minutes of Meeting	Meeting records with action points that can be tracked to closure.
Tender Dashboard	Pipeline view of projects by tender sub-stage, with bulk stage transfer.
Alerts	Warns when a PBG (bank guarantee) expires within 30 days, or an O&M contract is ending.
User Management	Create users, set role, grant per-user create/update/delete rights, assign divisions.
Audit Trail	Records every create, update and delete — who, when, and which fields changed.

> **One-line summary to open with:** "It is a role-based project monitoring dashboard for government infrastructure projects — a React front end, a Node/Express API, and a PostgreSQL database — that gives management a live view of progress, money and risk, with a full audit trail."

Architecture & Technology Stack

How the pieces fit together, and what each technology is for

3. System architecture



How the parts talk to each other

- The frontend is a **static build** (HTML, CSS, JS). Vercel serves it from a CDN — there is no React code running on a server.
- The browser calls **/api/…** on its own domain. Vercel **rewrites** those calls to the Render backend.
- Because the call looks same-origin to the browser, there is no CORS preflight, cookies flow normally, and the backend URL is never exposed in the UI.
- The backend is a **REST API returning JSON**. It never returns HTML.
- Express talks to PostgreSQL through **Drizzle ORM**, a type-safe query builder.

4. Technology stack

Frontend

React 18	Builds the user interface out of reusable components.
TypeScript	Adds types to JavaScript so mistakes are caught while coding, not in production.
Vite	Dev server and build tool. Very fast startup and hot reload.
Redux Toolkit + RTK Query	Holds app state (like the logged-in user) and handles all API calls, caching and refetching.
React Router	Moves between pages without reloading the browser.
Tailwind CSS	Styling with utility classes, so the look stays consistent.
Recharts	Draws the dashboard bar charts and donut charts.
ExcelJS / jsPDF / PptxGenJS	Exports the project table to Excel, PDF and PowerPoint; ExcelJS also reads the import file.
Zod	Checks form and environment values are the right shape.

Backend

Node.js + Express	The web server that receives HTTP requests and returns JSON.
TypeScript	Same safety benefit as on the frontend; shared mental model.
Drizzle ORM	Builds SQL queries in a type-safe way and manages migrations.
Zod	Validates every incoming request body and query string at the API edge.
jsonwebtoken	Creates and verifies the JWT access tokens used for login.
bcryptjs	Hashes passwords so they are never stored in plain text.
Helmet & CORS	Sets protective HTTP headers and controls which websites may call the API.
Nodemailer	Sends the password-reset OTP email.

Database

PostgreSQL	The main relational database — managed Postgres in the cloud for production, local Postgres for development.
SQL views	Pre-compute alert lists for the dashboard.

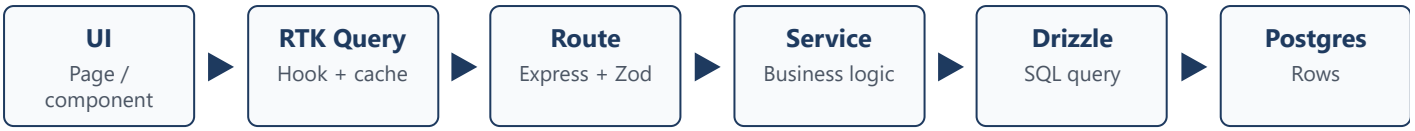
Dev & testing tools

Vitest	Unit tests (101 passing).
Playwright	End-to-end browser tests.
ESLint	Keeps code style consistent.

Code Flow — Request and Response

What actually happens when a user clicks something

5. The journey of one request



Example: opening the Projects page

1. User clicks **Projects** in the sidebar. React Router renders ProjectsPage.
2. The page calls a hook — useListProjectsQuery(filters).
3. RTK Query checks its **cache**. If the data is fresh it renders immediately and stops here.
4. Otherwise it calls baseQuery, which attaches **Authorization: Bearer <access token>** from the Redux store.
5. The browser sends GET /api/projects. Vercel rewrites it to the Render backend.
6. Express runs its middleware chain: **Helmet → CORS → JSON parser → cookie parser**.
7. **requireAuth** verifies the JWT signature and expiry, then attaches req.user.
8. A **permission gate** runs (for example requireProjectCreate). MD bypasses all gates; other roles need the flag on their account.
9. **Zod** parses the query string. Invalid input returns **400** before the database is touched.
10. The **service layer** (projectsService) holds the business logic and builds the query.
11. **Drizzle ORM** turns that into SQL. If the user is a PD, a division filter is added so they only see their own division.
12. PostgreSQL returns rows; the service shapes them into clean JSON.
13. The route responds { items, total }.
14. RTK Query stores the result under a **cache tag** (ProjectList) and the component re-renders with data.
15. Later, any create/update/delete **invalidates that tag**, so the list refetches automatically — no manual refresh code.

6. Login and session flow

1. User submits username and password to POST /api/auth/login.
2. The server finds the user and compares the password against the **bcrypt hash**.
3. On success it issues two tokens: a short-lived **access token** (JWT, 15 min) returned in the JSON body, and a long-lived **refresh token** (30 days) set as an **httpOnly cookie**.
4. The access token is kept in Redux memory — never in localStorage, so a script on the page cannot steal it.
5. When the access token expires, an API call returns **401**.
6. baseQuery catches that, calls /api/auth/refresh **once**, gets a new access token, and retries the original request automatically. The user notices nothing.
7. If the refresh itself is rejected, credentials are cleared and the user is sent to the login page.

> **Nice detail to mention:** refresh tokens are **rotated** — each use issues a new one and retires the old. Reuse of a retired token is treated as theft. A short grace window stops two browser tabs refreshing at the same moment from logging each other out.

Standard error format

Every failure returns the same shape, so the UI can handle errors in one place:
{ "error": { "code": "VALIDATION_ERROR", "message": "...", "details": { ... } } }

Modules, Database & API

How the data is organised and how the code is layered

7. Backend layering

Route
HTTP, auth gate, Zod validation

▶

Service
Business rules, permissions, audit

▶

Drizzle / DB
Type-safe SQL, transactions

Why it matters: routes stay thin and contain no SQL; services can be unit-tested without starting a web server; and the database layer can change without touching the API surface.

8. Database design (23 tables)

Group	Tables	Purpose
Master data	region, division, district, sector, scheme	Fixed reference lists used in dropdowns.
Core	project, project_scheme	The central project record and its many-to-many link to schemes.
Project details	project_milestone, milestone_progress, cos_eot_item, management_action_item, geo_photo, project_funds_uc, pre_monsoon_item	Child records that hang off a project.
Meetings	minutes_of_meeting, mom_action_point	Meeting notes and the actions agreed in them.
Users & security	app_user, user_division, refresh_token, password_reset_otp, password_reset_request	Accounts, division assignment, sessions and password recovery.
Audit	audit_log, audit_log_change	One row per action, plus one row per changed field.

Views: v_pbg_expiry_alerts and v_om_status are SQL views that pre-compute "expiring within 30 days" lists, so the dashboard reads one simple query instead of calculating in the browser. **Migrations** are numbered .sql files applied in order — the schema history is version-controlled.

9. API surface

/api/auth	login, logout, refresh, me, forgot-password OTP, reset password
/api/projects	list, read, create, update, delete, bulk Excel import, tender transfer; child routes for milestones, CoS/EoT, actions, photos
/api/kpis	dashboard totals, scheme chart, PBG expiry alerts, O&M status
/api/lookups	sectors, divisions, regions, districts, schemes for dropdowns
/api/users	user administration and permission flags
/api/mom, /funds-uc, /pre-monsoon	Minutes of meeting, fund utilisation, pre-monsoon checklist
/api/audit	Read the audit trail
/api/uploads, /api/health	File uploads and a health check used for monitoring

10. Two features worth explaining in depth

Excel import

- User downloads a template whose columns are generated from the same field list the API uses.
- The filled file is parsed **in the browser** with ExcelJS and every row is checked.
- Errors are shown per row. **If any row is invalid, nothing is imported** — it is all-or-nothing.
- Valid rows are posted to the API, which **re-validates with the same Zod schema** and inserts them in one transaction.

Audit trail

- Services record an audit entry whenever a record is created, updated or deleted.
- audit_log stores who, what and when; audit_log_change stores old and new values per field.
- This makes the system defensible in a government review — every number can be traced back.

Security, Deployment & Likely Questions

The last page to revise before you present

11. Security

Password storage	Hashed with **bcrypt**. Plain passwords are never stored or logged.
Authentication	Short-lived **JWT access token** in memory + rotating **refresh token** in an httpOnly cookie.
Token theft defence	Refresh tokens rotate on every use; reuse of an old token is detected and the chain is revoked.
Authorisation (RBAC)	Four roles — **MD, Admin, PD, Viewer** — plus per-user flags for create / update / delete. MD bypasses all flags.
Data scoping	A **PD only sees their own division**. Requests for other divisions return 404, so the existence of other data is not leaked.
Input validation	**Zod** validates every request at the API edge — bad data is rejected with 400 before reaching the database.
HTTP hardening	**Helmet** security headers and a **CORS allowlist** of permitted origins.
Rate limiting	**Upstash Redis** limits login attempts and uploads to slow brute-force attacks.
Accountability	Full audit log of every change, with field-level detail.

Important point to make: hiding a button in the UI is only cosmetic. Every rule is enforced again on the server, because the API is what actually protects the data.

12. Deployment

Frontend	**Vercel** — auto-deploys on every push to main. Live at buidco.vercel.app.
Backend	**Render** — runs the compiled Express server. Reached through a Vercel rewrite of /api/*.
Database	**PostgreSQL** — a managed cloud Postgres instance in production; a local Postgres server for development.
Config	All secrets come from environment variables — nothing sensitive is committed to git.

> **Known limitation, good to admit:** the backend runs on Render's free tier, which sleeps after about 15 minutes idle. The first request then takes roughly 35 seconds. Fix: a scheduled ping to keep it warm, or a paid instance that never sleeps.

13. Questions you may be asked

Why route API calls through Vercel instead of calling Render directly? — It makes the call same-origin, so there is no CORS preflight, cookies work naturally, and the backend URL stays hidden.

Why two tokens instead of one? — The access token is short-lived, so a stolen one is useless quickly. The refresh token is httpOnly, so JavaScript cannot read it.

Where is role-based access actually enforced? — In backend middleware. The UI also hides controls, but that is only for convenience; the server is the real gate.

Why Drizzle ORM and not raw SQL or Prisma? — It is type-safe and lightweight, reads close to SQL, and its migrations are plain .sql files we can review.

How do you stop bad data getting in? — Zod validates at the API edge, and the Excel import reuses the very same schema, so bulk and manual entry obey identical rules.

How are the dashboard numbers calculated? — With SQL aggregates and database views, not in the browser. That keeps it fast and consistent for everyone.

What happens if two tabs refresh the session at once? — Token rotation has a short grace window, so parallel refreshes do not log the user out.

How is the project tested? — 101 Vitest unit tests, Playwright end-to-end tests, and an automated audit suite that captures before/after evidence.

How would you scale this? — Database indexes on foreign keys are already in place; next steps are connection pooling, cached KPI queries, and a paid always-on backend.

What was the hardest part? — Session handling. Users were being logged out when a request merely timed out, so we separated “server said no” from “server did not answer” and only ended the session for the first.